09
Generici
(polimorfismo parametrico)

Mirko Viroli
mirko.viroli@unibo.it

C.D.L. Ingegneria e Scienze Informatiche
Alma Mater Studiorum—Università di Bologna, Cesena

a.a. 2020/2021

Outline

Goal della lezione

- Illustrare il problema delle collezioni polimorfiche
- Discutere il concetto di polimorfismo parametrico
- Illustrare i Generici di Java e alcuni loro vari dettagli

Argomenti

- Collezioni con polimorfismo inclusivo
- Classi generiche
- Interfacce generiche
- Metodi generici

Outline

1. Collections con polimorfismo inclusivo
2. Classi generiche
3. Interfacce generiche
4. Metodi generici
5. Java Wildcards

Forme di riuso nella programmazione OO

Composizione (e come caso particolare, delegazione)

Un oggetto è ottenuto per composizione di oggetti di altre classi

Estensione

Una nuova classe è ottenuta riusando il codice di una classe pre-esistente

Polimorfismo inclusivo (subtyping)

Una funzionalità realizzata per lavorare su valori/oggetti del tipo `A`, può lavorare con qualunque valore/oggetto del sottotipo `B` (p.e., se `B` estende la classe `A`, o se `B` implementa l'interfaccia `A`)

Polimorfismo parametrico (Java/C# generics, C++ templates,..)

Una funzionalità (classe o metodo) generica è costruita in modo tale da lavorare uniformemente su valori/oggetti indipendentemente dal loro tipo: tale tipo diventa quindi una sorta di parametro addizionale

Astrazioni uniformi con le classi

Astrazioni uniformi per problemi ricorrenti

- Durante lo sviluppo di vari sistemi si incontrano problemi ricorrenti che possono trovare una soluzione comune
- In alcuni casi queste soluzioni sono fattorizzabili (per astrazione) in una singola classe altamente riusabile

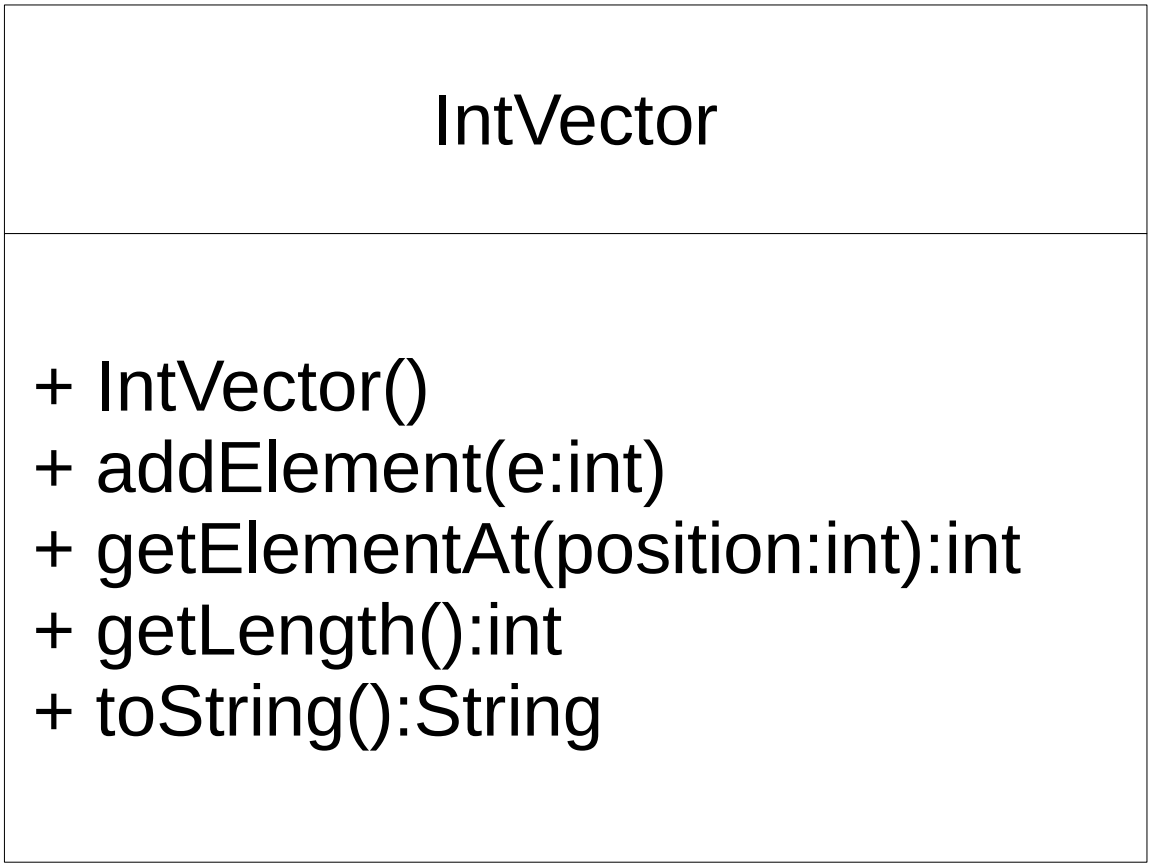
Un caso fondamentale: le collection

- Una collection è un oggetto il cui compito è quello di immagazzinare il riferimento ad un numero (tipicamente non precisato) di altri oggetti
- Fra i suoi compiti c'è quello di consentire modifiche ed accessi veloci all'insieme di elementi di tale collezioni
- Varie strategie possono essere utilizzate, seguendo la teoria/pratica degli algoritmi e delle strutture dati

Un esempio: IntVector

Collection IntVector

- Contiene serie numeriche (vettori) di dimensione non nota a priori
- Ossia, a lunghezza variabile..



UseIntVector

```
public class UseIntVector {
  public static void main(String[] s) {
    // Serie di Fibonacci
    final IntVector vi = new IntVector();
    // fib(0)=fib(1)=1, fib(N)=fib(N-1)+fib(N-2) if N>1
    vi.addElement(1);
    vi.addElement(1);
    for (int i = 0; i < 20; i++) {
      vi.addElement(vi.getElementAt(vi.getLength() - 1) + // ultimo
          vi.getElementAt(vi.getLength() - 2) // penultimo
      );
    }
    System.out.println(vi);
    // |1|1|2|3|5|8|13|21|34|55|89|144|233|..
    // 377|610|987|1597|2584|4181|6765|10946|17711|
  }
}
```

IntVector – implementazione

Collection IntVector

- Contiene serie numeriche (vettori) di dimensione non nota a priori
- Realizzata componendo un array che viene espanso all'occorrenza

IntVector
- INITIAL_SIZE: int - elements:int[] - size:int
+ IntVector() + addElement(e:int) + getElementAt(position:int):int - expand() + getLength():int + toString():String

IntVector pt 1

```java
public class IntVector{

    private static final int INITIAL_SIZE = 10;

    private int[] elements; // Deposito per gli elementi
    private int size;   // Numero di elementi

    public IntVector(){   // Inizialmente vuoto
      this.elements = new int[INITIAL_SIZE];
      this.size = 0;
    }

    public void addElement(final int e){
      if (this.size == elements.length){
          this.expand();  // Se non c'è più spazio..
      }
      this.elements[this.size] = e;
      this.size++;
    }

    public int getElementAt(final int position){
      return this.elements[position];
    }
```

IntVector pt 2

```java

    public int getLength(){
      return this.size;
    }

    private void expand(){  // Raddoppio lo spazio..
      final int[] newElements = new int[this.elements.length*2];
      for (int i=0; i < this.elements.length; i++){
          newElements[i] = this.elements[i];
      }
      this.elements = newElements;
      //this.elements = java.util.Arrays.copyOf(this.elements, this.elements.length*2);
    }

    public String toString(){
      String s="|";
      for (int i=0; i < size; i++){
          s = s + this.elements[i] + "|";
      }
      return s;
    }
}
```

Un primo passo verso l'uniformità

Solo elenchi di `int`?

- L'esperienza porterebbe subito alla necessità di progettare vettori di `float`, `double`, `boolean`,.. ossia di ogni tipo primitivo
- E poi, anche vettori di `String`, `Date`, eccetera
- L'implementazione sarebbe analoga, ma senza possibilità di riuso..

Collection uniformi "monomorfiche"

- Una prima soluzione del problema la si ottiene sfruttando il polimorfismo inclusivo e la filosofia "everything is an object" (incluso l'uso dell'autoboxing dei tipi primitivi)
- Si realizza unicamente un `ObjectVector`, semplicemente sostituendo `int` con `Object`
- Si inserisce qualunque elemento (via upcast implicito)
- Quando si riottiene un valore serve un downcast esplicito

Da IntVector a ObjectVector

IntVector
- INITIAL_SIZE: int - elements:int[] - size:int
+ IntVector() + addElement(e:int) + getElementAt(position:int):int - expand() + getLength():int + toString():String

ObjectVector
- INITIAL_SIZE: int - elements:Object[] - size:int
+ ObjectVector() + addElement(e:Object) + getElementAt(position:int):Object - expand() + getLength():int + toString():String

UseObjectVector

```java
public class UseObjectVector {
  public static void main(String[] s) {
    // Serie di Fibonacci
    final ObjectVector vobj = new ObjectVector();
    // fib(0)=fib(1)=1, fib(N)=fib(N-1)+fib(N-2) if N>1
    vobj.addElement(1); // grazie all'autoboxing
    vobj.addElement(1);
    for (int i = 0; i < 20; i++) {
      vobj.addElement( // servono downcast specifici
      (Integer) vobj.getElementAt(vobj.getLength() - 1)
          + (Integer) vobj.getElementAt(vobj.getLength() - 2));
    }
    System.out.println(vobj);
    // |1|1|2|3|5|8|13|21|34|55|89|144|233|..
    // 377|610|987|1597|2584|4181|6765|10946|17711|

    // Altro esempio
    final ObjectVector vobj2 = new ObjectVector();
    vobj2.addElement("Prova");
    vobj2.addElement("di");
    vobj2.addElement("vettore");
    vobj2.addElement(new Object());
    System.out.println(vobj2);
    String str = (String) vobj2.getElementAt(1); // "di"
    // String str2 = (String)vobj2.getElementAt(3); // Exception
  }
}
```

Un altro caso di collection, la list linkata ObjectList

```java
/* Lista linkata di oggetti, con soli metodi getter */
public class ObjectList {

  private final Object head;
  private final ObjectList tail;

  public ObjectList(final Object head, final ObjectList tail) {
    this.head = head;
    this.tail = tail;
  }

  public Object getHead() { // Testa della lista
    return this.head;
  }

  public ObjectList getTail() { // Coda della lista
    return this.tail;
  }

  public int getLength() { // Dimensione della lista
    return (this.tail == null) ? 1 : 1 + this.tail.getLength();
  }

  public String toString() { // Rappr. a stringa
    return "|" + this.head
        + ((this.tail == null) ? "|" : this.tail.toString());
  }
}
```

UseObjectList

```java
public class UseObjectList {
  public static void main(String[] s) {
    final ObjectList list =
        new ObjectList(10, new ObjectList(20,
        new ObjectList(30, new ObjectList(40, null))));
    // Cast necessari, eccezioni possibili
    final int first = (Integer) list.getHead(); // Unboxing
    final int second = (Integer) list.getTail().getHead();
    final int third = (Integer) list.getTail().getTail().getHead();
    System.out.println(first + " " + second + " " + third);
    System.out.println(list.toString());
    System.out.println(list.getLength());

    // Usabile anche con le stringhe
    final ObjectList list2 = new ObjectList("a",
        new ObjectList("b",
        new ObjectList("c",
        new ObjectList("d", null))));
    System.out.println(list2.toString());
  }
}
```

La necessità di un approccio a polimorfismo parametrico

Prima di Java 5

- Questo era l'approccio standard alla costruzione di collection
- Java Collection Framework — una libreria fondamentale

Problema

- Con questo approccio, nel codice Java risultavano molti usi di oggetti simili a `ObjectVector` o `ObjectList`
- Si perdeva molto facilmente traccia di quale fosse il contenuto..
  - contenevano oggetti vari? solo degli `Integer`? solo delle `String`?
- Il codice conteneva spesso dei downcast sbagliati, e quindi molte applicazioni Java fallivano generando dei `ClassCastException`

Più in generale

Il problema si manifesta ogni volta che voglio collezionare oggetti il cui tipo non è noto a priori, ma potrebbe essere soggetto a polimorfismo inclusivo

Polimorfismo parametrico

Idea di base

- Dato un frammento di codice `F` che lavora su un certo tipo, diciamo `String`, se potrebbe anche lavorare in modo uniforme con altri..
- ..lo si rende parametrico, sostituendo a `String` una sorta di variabile `X` (chiamata type-variable, ossia una variabile che contiene un tipo)
- A questo punto, quando serve il frammento di codice istanziato sulle stringhe, si usa `F<String>`, ossia si richiede che `X` diventi `String`
- Quando serve il frammento di codice istanziato sugli integer, si usa `F<Integer>`

Java Generics

- Classi/interfacce/metodi generici
- Nessun impatto a run-time, per via dell'implementazione a "erasure"
  - `javac` "compila via i generici", quindi la JVM non li vede

Outline

1. Collections con polimorfismo inclusivo
2. Classi generiche
3. Interfacce generiche
4. Metodi generici
5. Java Wildcards

La classe generica List

```
/* Classe generica in X:
   - X è il tipo degli elementi della lista */
public class List<X>{

    private final X head;     // Testa della lista, tipo X
    private final List<X> tail;  // Coda della lista, tipo List<X>

    public List(final X head, final List<X> tail){
      this.head = head;
      this.tail = tail;
    }

    public X getHead(){
      return this.head;
    }

    public List<X> getTail(){
      return this.tail;
    }

    // getLength() e toString() invariate
    ...
}
```

Uso di una classe generica

```
public class UseList{
    public static void main(String[] s){
      final List<Integer> list =
          new List<Integer>(10, // Autoboxing
            new List<Integer>(20,
            new List<Integer>(30,
            new List<Integer>(40,null))));
      // Cast NON necessari
      final int first = list.getHead(); // Unboxing
      final int second = list.getTail().getHead();
      final int third = list.getTail().getTail().getHead();
      System.out.println(first+" "+second+" "+third);
      System.out.println(list.toString());
      System.out.println(list.getLength());

      // Usabile anche con le stringhe
      final List<String> list2 = new List<String>("a",
                    new List<String>("b",
                    new List<String>("c",
                    new List<String>("d",null))));
      System.out.println(list2.toString());
    }
}
```

Terminologia, e elementi essenziali

Data una classe generica `C<X,Y>`..

- `X` e `Y` sono dette le sue type-variable
- `X` e `Y` possono essere usati come un qualunque tipo dentro la classe (con alcune limitazioni che vedremo)

I clienti delle classi generiche

- Devono usare tipi generici: versioni "istanziate" delle classi generiche
  - `C<String,Integer>`, `C<C<Object,Object>,Object>`
  - Non `C` senza parametri, altrimenti vengono segnalati dei warning
- Ogni type-variable va sostituita con un tipo effettivo, ossia con un parametro, che può essere
  - una classe (non-generica), p.e. `Object`, `String`,..
  - una type-variable definita, p.e. `X,Y` (usate dentro la classe `C<X,Y>`)
  - un tipo generico completamente istanziato, p.e. `C<Object,Object>`
  - ..o parzialmente istanziato, p.e. `C<Object,X>` (in `C<X,Y>`)
  - NON con un tipo primitivo

La classe generica Vector

```
1  public class Vector<X>{
2  // X è la type-variable, ossia il tipo degli elementi
3      ...
4      public Vector(){...}
5
6      public void addElement(X e){...}  // Input di tipo X
7
8      public X getElementAt(int pos){...} // Output di tipo X
9
10     public int getLength(){...}
11
12     public String toString(){...}
13 }
```

Uso di Vector<X>

```
public class UseVector{
  public static void main(String[] s){

      // Il tipo di vs è Vector<String>
      // Ma la sua classe è Vector<X>
      final Vector<String> vs = new Vector<String>();
      vs.addElement("Prova");
      vs.addElement("di");
      vs.addElement("Vettore");
      final String str = vs.getElementAt(0) + " " +
             vs.getElementAt(1) + " " +
             vs.getElementAt(2); // Nota, nessun cast!
      System.out.println(str);

      final Vector<Integer> vi=new Vector<Integer>();
      vi.addElement(10); // Autoboxing
      vi.addElement(20);
      vi.addElement(30);
      final int i = vi.getElementAt(0) + // Unboxing
        vi.getElementAt(1) +
        vi.getElementAt(2);
      System.out.println(i);
    }
}
```

Implementazione di Vector pt 1

```
public class Vector<X>{

    private final static int INITIAL_SIZE = 10;

    private Object[] elements;  // No X[], devo usare Object[]!!
    private int size;

    public Vector(){
      this.elements = new Object[INITIAL_SIZE]; // Object[]
      this.size = 0;
    }

    public void addElement(X e){  // Tutto come atteso
      if (this.size == elements.length){
          this.expand();
      }
      this.elements[this.size] = e;
      this.size++;
    }
    ...
```

Implementazione di Vector pt 2

```
    ...
    public X getElementAt(int position){
      return (X)this.elements[position]; // Conversione a X
      // Genera un unchecked warning!
    }

    private void expand(){
      // Ancora Object[]
      Object[] newElements = new Object[this.elements.length*2];
      for (int i=0; i < this.elements.length; i++){
          newElements[i] = this.elements[i];
      }
      this.elements = newElements;
    }

    // getLength() e toString() inalterate
    ...
}
```

La classe generica Pair<X,Y>

```
public class Pair<X, Y> {

  private final X first;
  private final Y second;

  public Pair(final X first, final Y second) {
    this.first = first;
    this.second = second;
  }

  public X getFirst() {
    return this.first;
  }

  public Y getSecond() {
    return this.second;
  }

  public String toString() {
    return "<" + this.first + "," + this.second + ">";
  }
}
```

Uso di Pair<X,Y>

```
public class UsePair{
    public static void main(String[] s){
      Pair<String,Integer> p = new Pair<String,Integer>("aa",1);
      String fst = p.getFirst();
      int snd = p.getSecond();
      System.out.println(fst+" "+snd);

      final Vector<Pair<String,Integer>> v =
          new Vector<Pair<String,Integer>>();
      v.addElement(new Pair<String,Integer>("Prova",1));
      v.addElement(new Pair<String,Integer>("di",2));
      v.addElement(new Pair<String,Integer>("Vettore",3));
      final String str = v.getElementAt(0).getFirst() + " " +
             v.getElementAt(1).getFirst() + " " +
             v.getElementAt(2).getFirst(); // Nota, nessun cast!
      System.out.println(str);
      System.out.println(v);

      final List<Pair<Integer,Integer>> l =
          new List<Pair<Integer,Integer>>(new Pair<Integer,Integer>(1,1),
          new List<Pair<Integer,Integer>>(new Pair<Integer,Integer>(2,2),
          new List<Pair<Integer,Integer>>(new Pair<Integer,Integer>(3,3),
          null)));
      System.out.println(l);
    }
}
```

Inferenza dei parametri

Un problema sintattico dei generici

- Tendono a rendere il codice più pesante ("verbose")
- Obbligano a scrivere i parametri anche dove ovvi, con ripetizioni

L'algoritmo di type-inference nel compilatore

- Nella `new` si possono tentare di omettere i parametri (istanziazione delle type-variable), indicando il "diamond symbol" <>
- Il compilatore cerca di capire quali siano questi parametri guardando gli argomenti della `new` e l'eventuale contesto dentro il quale la `new` è posizionata, per esempio, se assegnata ad una variabile
- Nel raro caso in cui non ci riuscisse, segnalerebbe un errore a tempo di compilazione.. quindi tanto vale provare!
- Ricordarsi <>, altrimenti viene confuso con un raw type, un meccanismo usato per gestire il legacy con le versioni precedenti di Java

La local variable type inference

- in genere è alternativa al simbolo <>

Esempi di inferenza

```java
public class UsePair2{
    public static void main(String[] s){
      // Parametri in new Vector() inferiti dal tipo della variabile
      final Vector<Pair<String,Integer>> v = new Vector<>();
      // Parametri in new Pair(..) inferiti dal tipo degli argomenti
      v.addElement(new Pair<>("Prova",1));
      v.addElement(new Pair<>("di",2));
      v.addElement(new Pair<>("Vettore",3));
      final String str = v.getElementAt(0).getFirst() + " " +
             v.getElementAt(1).getFirst() + " " +
             v.getElementAt(2).getFirst(); // Nota, nessun cast!
      System.out.println(str);
      System.out.println(v);

        // Inferenza grazie agli argomenti e tipo variabile..
      final List<Pair<Integer,Integer>> l =
          new List<>(new Pair<>(1,1),
          new List<>(new Pair<>(2,2),
          new List<>(new Pair<>(3,3),
          null)));
      System.out.println(l);

      // Local variable type inference
      final var v2 = new Vector<Integer>();
      v2.addElement(1);
      System.out.println(v2);
    }
}
```

I vantaggi dei generici

Coi generici, Java diventa un linguaggio molto più espressivo!

Svantaggi

- Il linguaggio risulta più sofisticato, e quindi complesso
- Se non ben usati, possono minare la comprensibilità del software
- Non vanno abusati!!
- Gli aspetti più avanzati dei generici, che vedremo, sono considerati troppo complessi

Vantaggi – se ben usati

- Codice più comprensibile
- Maggiori possibilità di riuso di funzionalità (quasi d'obbligo oramai)
- Codice più sicuro (safe) – il compilatore segnala errori difficili da trovare altrimenti

Outline

1. Collections con polimorfismo inclusivo
2. Classi generiche
3. Interfacce generiche
4. Metodi generici
5. Java Wildcards

Interfacce generiche

Cosa è una interfaccia generica

- È una interfaccia che dichiara type-variables: `interface I<X,Y>{..}`
- Le type-variable compaiono nei metodi definiti dall'interfaccia
- Quando una classe la implementa deve istanziare le type variabile (o assegnarle ad altre type-variable se essa stessa è generica)

Utilizzi

Per creare contratti uniformi che non devono dipendere dai tipi utilizzati

Un esempio notevole, gli Iteratori

- Un iteratore è un oggetto usato per accedere ad una sequenza di elementi
- Ne vedremo ora una versione semplificata – diversa da quella delle librerie Java

L'interfaccia Iterator

```
public interface Iterator<E> {

  // torna il prossimo elemento dell'iterazione
  E next();

  // dice se vi saranno altri elementi
  boolean hasNext();

  /*
   * Nota: non è noto cosa succede se si chiama
   * next() quando hasNext() ha dato esito falso
   */
}
```

Implementazione 1: IntRangeIterator

```java
/* Itera tutti i numeri interi fra 'start' e 'stop' inclusi */
public class IntRangeIterator implements Iterator<Integer> {

  private int current; // valore corrente
  private final int stop; // valore finale

  public IntRangeIterator(final int start, final int stop) {
    this.current = start;
    this.stop = stop;
  }

  public Integer next() {
    return this.current++;
  }

  public boolean hasNext() {
    return this.current <= this.stop;
  }
}
```

Implementazione 2: ListIterator

```java
/* Itera tutti gli elementi di una List */
public class ListIterator<E> implements Iterator<E> {

  private List<E> list; // Lista corrente

  public ListIterator(final List<E> list) {
    this.list = list;
  }

  public E next() {
    final E element = this.list.getHead();// Elemento da tornare
    this.list = this.list.getTail();// Aggiorno la lista
    return element;
  }

  public boolean hasNext() {
    return (this.list != null);
  }
}
```

Implementazione 3: VectorIterator

```java
/* Itera tutti gli elementi di un Vector */
public class VectorIterator<E> implements Iterator<E> {

  private final Vector<E> vector; // Vettore da iterare
  private int current; // Posizione nel vettore

  public VectorIterator(final Vector<E> vector) {
    this.vector = vector;
    this.current = 0;
  }

  public E next() {
    return this.vector.getElementAt(this.current++);
  }

  public boolean hasNext() {
    return this.vector.getLength() > this.current;
  }
}
```

UseIterators: nota l'accesso uniforme!

```
public class UseIterators {
  public static void main(String[] s) {
    final List<String> list = new List<>("a",
                                 new List<>("b",
                                   new List<>("c", null)));

    final Vector<Date> vector = new Vector<>();
    vector.addElement(new Date());
    vector.addElement(new Date());

    // creo 3 iteratori..
    final Iterator<Integer> iterator = new IntRangeIterator(5, 10);
    final Iterator<String> iterator2 = new ListIterator<>(list);
    final Iterator<Date> iterator3 = new VectorIterator<>(vector);

    // ne stampo il contenuto..
    printAll(iterator);
    printAll(iterator2);
    printAll(iterator3);
  }

  static <X> void printAll(Iterator<X> iterator) {
    while (iterator.hasNext()) {
      System.out.println("Elemento : " + iterator.next());
    }
  }
}
```

Outline

1. Collections con polimorfismo inclusivo
2. Classi generiche
3. Interfacce generiche
4. Metodi generici
5. Java Wildcards

Metodi generici

Metodo generico

Un metodo che lavora su qualche argomento e/o valore di ritorno in modo independente dal suo tipo effettivo. Tale tipo viene quindi astratto in una type-variable del metodo.

Sintassi

- def: `<X1,..,Xn> ret-type nome-metodo(formal-args){...}`
- call: `receiver.<X1,..,Xn>nome-metodo(actual-args){...}`
- call con inferenza, stessa sintassi delle call standard, ossia senza <>

Due casi principali, con medesima gestione

- Metodo generico (statico o non-statico) in una classe non generica
- Metodo generico (non-statico) in una classe generica

⇒ Il primo dei due molto più comune..

Definizione di un metodo generico

```
import java.util.Date;
public class UseIterators2{

    public static <E> void printAll(Iterator<E> iterator){
      while (iterator.hasNext()){
          System.out.println("Elemento : "+iterator.next());
      }
    }

    ...
```

Chiamata a metodo generico

```
import java.util.Date;
public class UseIterators2{
    public static <E> void printAll(Iterator<E> iterator){..}

    public static void main(String[] s){
        Iterator<Integer> iterator = new IntRangeIterator(5,10);

        List<String> list = ..
        Iterator<String> iterator2 = new ListIterator(list);

        Vector<Date> vector= ..
        Iterator<Date> iterator3 = new VectorIterator(vector);

        // Attenzione, il nome della classe è obbligatorio
        UseIterators2.<Integer>printAll(iterator);
        UseIterators2.<String>printAll(iterator2);
        UseIterators2.<Date>printAll(iterator3);

        // Con inferenza, il nome della classe non è obbligatorio
        printAll(iterator);
        printAll(iterator2);
        printAll(iterator3);
    }
}
```

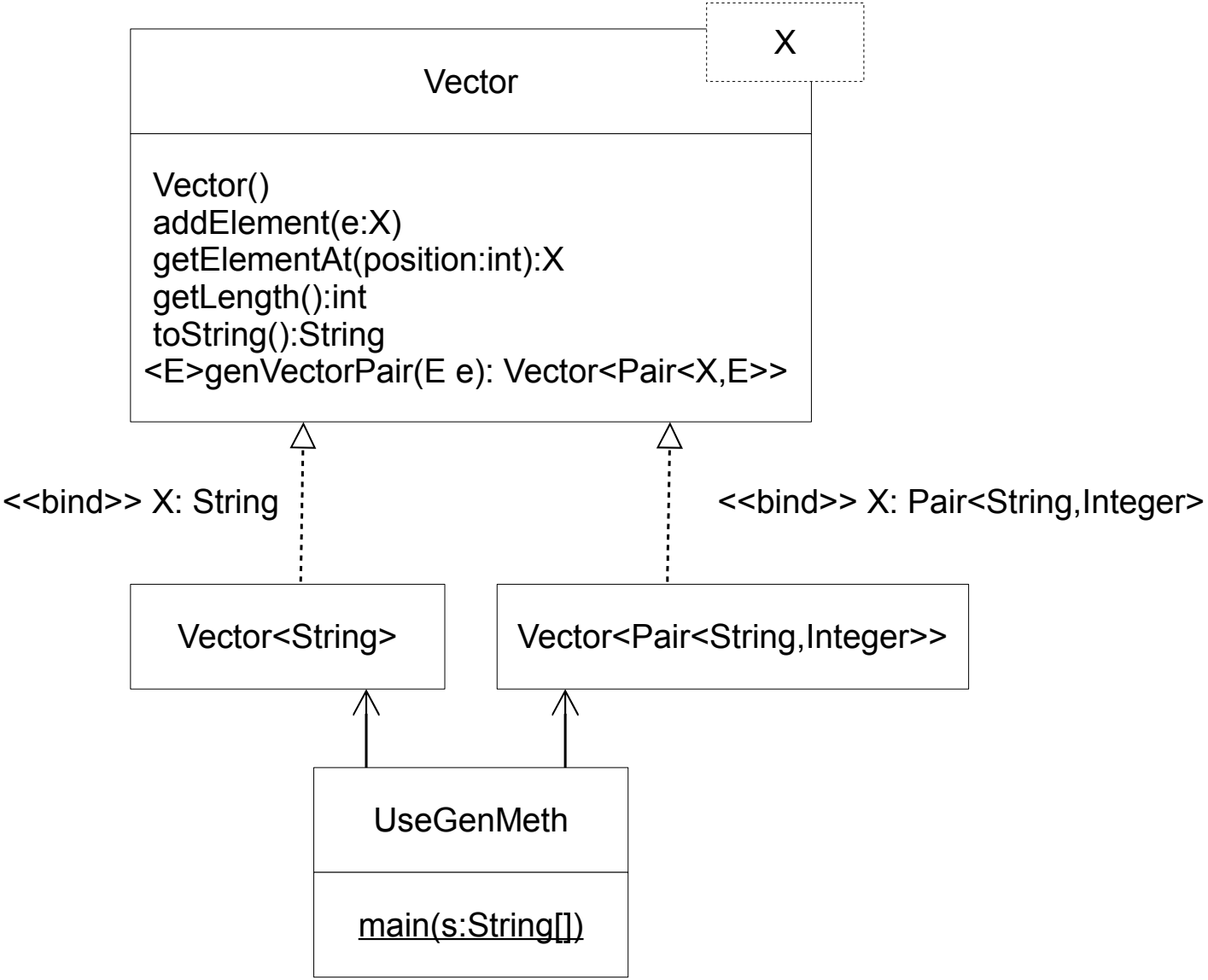
Esempio di metodo generico in classe generica

```
public class Vector<X>{

    ...
    <E> Vector<Pair<X,E>> genVectorPair(E e){
      Vector<Pair<X,E>> vp = new Vector<>(); /// Inferenza
      for (int i=0;i<this.size;i++){
          vp.addElement(new Pair<>(this.getElementAt(i),e));
      }
      return vp;
    }
}

public class UseGenMeth{
    public static void main(String[] s){
      Vector<String> vs = new Vector<>();
      vs.addElement("prova");
      vs.addElement("di");
      vs.addElement("vettore");
      Vector<Pair<String,Integer>> vp = vs.<Integer>genVectorPair(101);
      // versione con inferenza..
      // Vector<Pair<String,Integer>> vp2 = vs.genVectorPair(101);
      System.out.println(vp);
      // |<prova,101>|<di,101>|<vettore,101>|
    }
}
```

Notazione UML: non del tutto standard



Outline

1. Collections con polimorfismo inclusivo
2. Classi generiche
3. Interfacce generiche
4. Metodi generici
5. Java Wildcards

Java Wildcards

Osservazione

- Esistono situazioni in cui un metodo debba accettare come argomento non solo oggetti di un tipo `C<T>`, ma di ogni `C<S>` dove `S <: T`
- Esempio: un metodo statico `printAll()` che prende in ingresso un iteratore e ne stampa gli elementi
- È realizzabile con un metodo generico, ma ci sono casi in cui si vorrebbe esprimere un tipo che raccolga istanziazioni diverse di una classe generica

Java Wildcards

- Un meccanismo avanzato, quello inventato più di recente (2004-2006)
  - (Igarashi & Viroli) + (Bracha & Gafter) + (Torgersen & Hansen & von der Ahé)
- Fornisce dei nuovi tipi, chiamati Wildcards
- Simili a interfacce (non generano oggetti, descrivono solo contratti)
- Generalmente usati come tipo dell'argomento di metodi

Java Wildcards

```
// Gerarchia dei wrapper Numbers in java.lang
abstract class Number extends Object{...}
class Integer extends Number{...}
class Double extends Number{...}
class Long extends Number{...}
class Float extends Number{...}
...
```

```
// Accetta qualunque Vector<T> con T <: Number
// Vector<Integer>, Vector<Double>, Vector<Float>,...
void m(Vector<? extends Number> arg){...}

// Accetta qualunque Vector<T>
void m(Vector<?> arg){...}

// Accetta qualunque Vector<T> con Integer <: T
// Vector<Integer>, Vector<Number>, e Vector<Object> solo!
void m(Vector<? super Integer> arg){...}
```

Java Wildcards

3 tipi di wildcard

- Bounded (covariante): `C<? extends T>`
  - accetta un qualunque `C<S>` con `S <: T`
- Bounded (controvariante): `C<? super T>`
  - accetta un qualunque `C<S>` con `S >: T`
- Unbounded: `C<?>`
  - accetta un qualunque `C<S>`

Uso delle librerie che dichiarano tipi wildcard

- Piuttosto semplice, basta passare un argomento compatibile o si ha un errore a tempo di compilazione

Progettazione di librerie che usano tipi wildcard

- Molto avanzato: le wildcard pongono limiti alle operazioni che uno può eseguire, derivanti dal principio di sostituibilità

Esempio Wildcard

```
public class Wildcard{

  // Metodo che usa la wildcard
    public static void printAll(Iterator<?> it){
      while (it.hasNext()){
        System.out.println(it.next());
      }
    }

  // Analoga versione con metodo generico
    public static <T> void printAll2(Iterator<T> it){
      while (it.hasNext()){
        System.out.println(it.next());
      }
    }

    // Quale versione preferibile?

    public static void main(String[] s){
      Wildcard.printAll(new IntRangeIterator(1,5));
      Wildcard.printAll2(new IntRangeIterator(1,5));
      Wildcard.<Integer>printAll2(new IntRangeIterator(1,5));
    }
}
```